

ENG EC 311 Final Project Report - Frogger

Nathan Kim, Yixuan Huang, Charles Westwood, Hoang Yen Tran

December 10, 2025

1 Abstract

In this paper, the objective of this project is to create a successful Frogger game, originated in 1981. The logic system had to be designed in order to handle user inputs, along with many conditions based on the VGA display. This project was divided into four different parts. The first part involves scene generation and object movement intervals, utilizing VGA (Video Graphics Array) synchronized counters, along with timed intervals for car, log, and turtle position. The second part is responsible for the memory of the artwork. This was achieved through the built in block RAM (Random Access Memory) due to the textures created, creating an efficient memory block that can retrieve pixel textures and sprite data. The third part is responsible for collision detection, through state machines and combinational logic, determining safe zones. The fourth part is responsible for user controlled movement, while embedding a debouncer for reliable movement. The part also needs to update the frog location and ensures accurate location, whenever the frog may be on the screen. Together with the four parts, Frogger is able to be successfully implemented on the FPGA (Field-Programmable Gate Array), which is fully synchronized with all the parts.

2 Introduction

In 1984, the first Verilog was created by Phil Moorby and Prabhu Goel at Gateway Design Automation. It was first commercially used in 1985, and used to describe complex digital circuits in 1995 by IEEE. Today, Verilog is taught in institutions internationally because of the HDL (Hardware Description Language) it offers, describing, simulating, synthesizing digital circuits. In an FPGA, the internal hardware can be modified by the user to implement the circuit through the HDL. This is done with a physical chip embedded with the FPGA, specifically the Xilinx Artix-7 xc7a100tcsg324-1¹. The VGA standard is introduced by IBM in 1987, and the technology is utilized with the FPGA, at a 60 Hz refresh rate. The standard VGA format for FPGAs are 640 columns by 480 rows. The format includes multiple intervals, such as horizontal sync, vertical sync, front-porch and back-porch. The porch regions were utilized in order to have the objects wrap across the display, which will be discussed later in this paper². Concepts taken from ENG EC 311 at Boston University is utilized in order to further develop algorithms and hardware designing, such as combinational circuits, multiplexers, memory, counters, and state machine, and flip flop is the backbone for this entire project³.

3 Overview

In order to satisfactorily complete this project in a timely manner, work was split into various subsections: Object Generation, Graphics & Textures, Collision, and Game Logic. Each section was developed around agreed specifications such as encoding type and bus width such that independently authored modules would function cohesively once integrated into a single hardware implementation. Additionally, each module was tested either by temporarily hard-coding inputs, using simulation, or displaying outputs such that the correct operation of all logic could be verified.

¹See Digilent Inc.

²See Patel and Mishra

³See Boston University



Figure 1: Title Screen Displayed on a Cathode Ray Tube

3.1 Object Generation

The Object Generation module created objects and moves it along with screen based on the global clock signal. During each cycle created through clock division, it updates the object's horizontal position, and created a wrap-around behavior when objects reaches a value that is outside of the visual display area. This allows a smooth and continuous motion being seen by the user as they play the game.

3.2 Graphics & Textures

The Graphics and Textures modules are responsible for mapping textures, overlays and glyphs to screen pixels. To do so, combinational logic was used to determine the correct output value from a ROM, which was then output to the calling module, and from there the highest priority layer with a color value was printed to the display.

3.3 Collision Detection

The collision detect module sends a collision type signal to movement module to update lives and scores. It receives the game object (cars and logs) position from the level generation module.

3.4 Game Logic

This module manages the frog's state and interactions with its surroundings. By continuous tracking of position and processing signal from collision module, both the game and the frogs' position can be classified into well-defined states. These states determine rewards, remaining lives, timer, and movement's behavior.

4 Implementation

4.1 Implementing VGA

To start off the project with a proper VGA display, the VGA needs to be understood and programmed based on the RGB value, each color is 16 different values. There is a total combination of 16^3 . Each RGB value is assigned, specially based on the object that is being created, or the background color. The background color being responsible will be the line control color. Each line will have a definite color in the background, and ensuring everyone understands where the graphics and logics should be placed at. The object creation is divided into three parts. Creating multiple objects, ensuring all the lines can have two through five objects at once. The objects are equally spaced out, in accordance with the original frogger. Secondly, the object wraps around the screen. It is able to do that because the calculation is done within the porch area. The ensures the user does not see, or get confused when playing the frogger game.

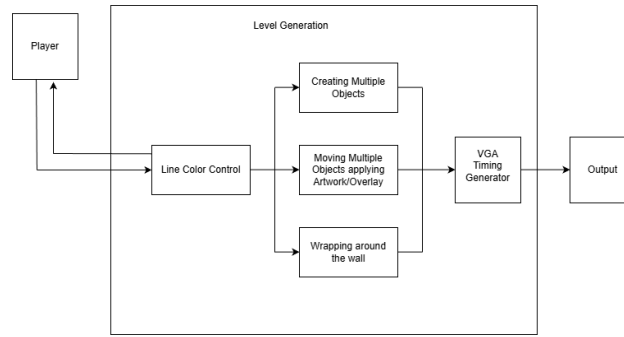


Figure 2: Logic Used to create the basics of frogger

4.1.1 VGA Jitter

This is one of the biggest issues the team has faced throughout the project. Notable jittering has been showing since the implementation of objects as they interact with the boundary of the edge. The biggest reason why is because of the pixel clock that is being utilized (25 MHz) while the expected rate pixel clock is different (25.175 MHz).

$$\Delta T_{\text{pixel}} = \frac{1}{25,175,000} - \frac{1}{25,000,000} = 40 \text{ ns} - 39.7 \text{ ns} = 0.278 \text{ ns} \quad (1)$$

$$\Delta T_{\text{error}} = T_{\text{FPGA}} - T_{\text{ideal}} = (800 \times 40 \text{ ns}) - (800 \times 39.7 \text{ ns}) = 0.223 \mu\text{s} \quad (2)$$

$$T_{\text{pixel,error}} = \frac{0.223 \mu\text{s}}{39.7 \text{ ns}} \approx 5.614 \text{ pixels per frame drift} \quad (3)$$

This leads to a 0.7% difference, which is higher than the clock tolerance that the VESA pixel clock can tolerate⁴. The total drift that occurs because the drift is 223 μs with a 5.6 pixels drift per frame. In order to fix this issue, a black "buffer" region was created in the active video area. This allows the jitter to stay hidden, while creating the stabilizing margin. However, it is important to note that the objects continue to be calculated by the system, but they are not displayed while the VGA signal is in the porch intervals.

4.2 Object Implementation

The objects created in this subsection complements the work described in Section 4.1. This was achieved by implemented modular bounding boxes. This method provides a structured way to represent each game element and offers flexibility for future graphical enhancements (see Section 4.3 and beyond). While the VGA module in Section 4.1 handles the display of the modular bounding boxes and the movement, the

⁴See Green

underlying object locations is defined to support later subsection's work such as subsection 4.3 and beyond. To accomplish this, each row containing moving objects outputs its corresponding $(x, y) = (\text{horizontal}, \text{vertical})$. The thickness and the height of the objects were outputs as well. These outputs allow the object-generation logic to communicate effectively with downstream modules, ensuring that collision detection, graphical overlays, and texture mapping operate correctly with respect to the spatial boundaries defined by each bounding box.

4.3 Graphics Implementation

In order to maximize player experience and create the most visually appealing game possible, any hardware engineer must take significant efforts to maximize the quality of textures, maximize interactivity, and provide high-quality feedback. In order to do this, high-quality sprites must be rendered. For this project, this was achieved by using Microsoft Paint to draw individual textures and sprites, paying special attention to hardware constraints, such as making sure that the texture dimensions are 2^n so that modulo and bit-shifting can be used to prevent the use of slow and large divider logic. Additionally, the color scheme had to be carefully selected to maximize visual appeal and work within the strict 12-bit color constraints of the hardware. A selection of artwork created for this project is provided below.

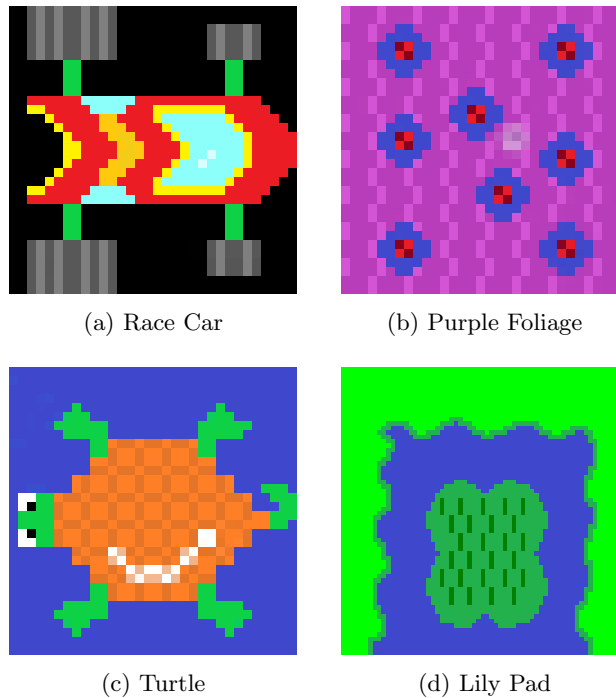


Figure 3: Artwork Used in the Project

Once a texture was drawn, it was exported in full-color Bitmap format and a Python script using the Python Image Library (PIL) was used to output the RGB colors of each pixel which were truncated to four bits per color and written to a .mem file in which each column represented a unique color channel. Within the .mem file, each four-bit nibble representing a pixel's R, G or B value was represented as a hexadecimal character between 0 and F, and for solid-color overlays, a 1 was used to indicate transparency while a 0 indicated that the pixel should be colored. Then, this file was imported into Vivado using the `$readmemh()` command.⁵ The file was structured such that when it was imported into Vivado, it would structure a ROM with a unique address for each pixel, each of which stores all twelve bits of color data.⁶ Once loaded into a ROM, a simple adder and bit shifter were used to convert the mapped x and y coordinates of the desired

⁵See Xilinx, Inc., Vivado Design Suite User Guide: Synthesis (UG901, v2022.2)

⁶See Project F

pixel color value to a ROM memory address, and when this address was read, the RGB values were passed directly to the calling module. An example of a simple texture signal generator is provided below:

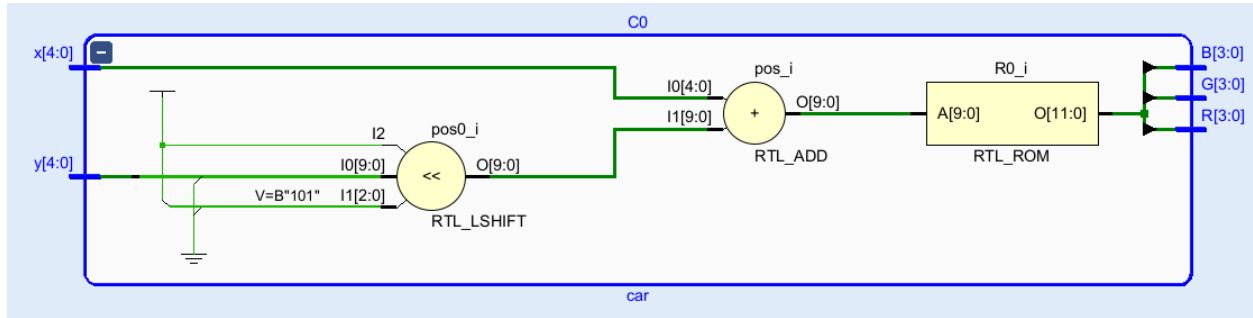


Figure 4: Logic Used to Render Race Car Sprites

4.3.1 Non-Static Graphics

For modules responsible for returning feedback to the player, such as the scoreboard, or those featuring animations, it was necessary to devise a means of toggling between multiple ROMs to display the correct pixel data at the correct time. To accomplish this, the interactive modules took in a control signal specifying what texture to output, and from there, a combination of muxes would select which ROM to output, or the means by which the ROM data should be traversed. For example, while the frog sprite has four possible orientations, only a single ROM was instantiated since the same pixel data can be read in different orders to produce the desired rotation. For the BCD counters, individual ROMs were created for each digit, and a digit-specific MUX selected which number to display while an overarching multi-digit controller selected which digit to output based on the current pixel coordinates. As an example of how non-standard graphics were implemented, a schematic of the frog module has been provided below.

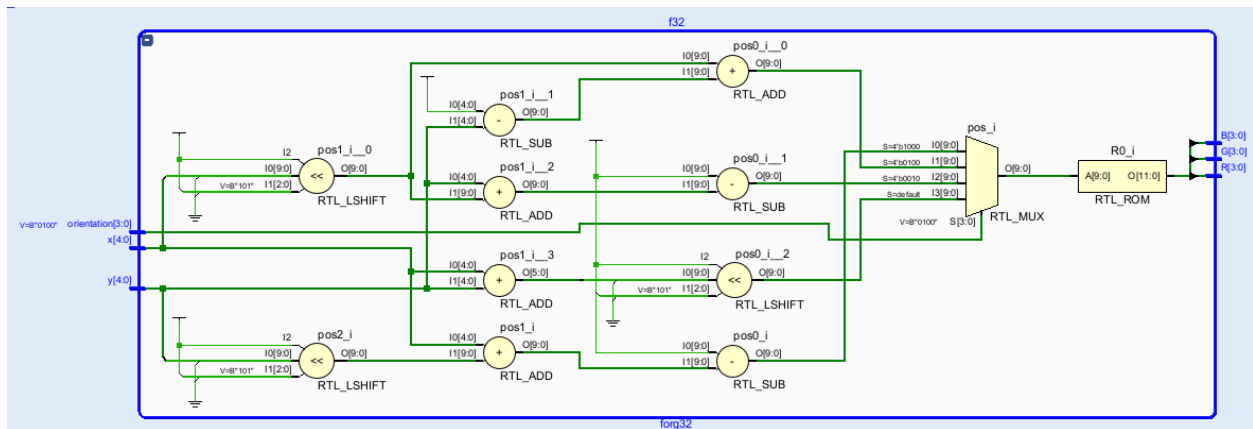


Figure 5: Frog Logic Schematic

4.3.2 Overlay Masking & BRAM Use

In order to provide game information, as well as count down to the start of the game, it was necessary that an overlay layer be created such that large text could be displayed. However, this came with numerous challenges, particularly the fact that storing eight twelve-bit images in ROMs would take up an enormous number of flip-flops and lookup tables, much more than was available on the chip. As such, it was necessary to optimize the image. To do so, the image was stored in a monochrome format with a 1 representing transparency and a 0 representing a colored pixel. Since all of the text was of the same color, minimal information loss resulted, but the issue still remained that the text data were too large to be stored in a

similar fashion to the sprites. To solve this, Block Random Access Memory, or BRAM for short, was used.⁷ By storing the pixel data for each image as one bit of an eight-bit word in BRAM, an enormous reduction in overall logic complexity resulted. By passing all eight bits from BRAM to a flip flop, and from there using a mux to select the desired output value, it was possible to store and retrieve multiple full-resolution text overlays, something that would have been impossible otherwise. A schematic is provided below to illustrate how BRAM was successfully instantiated for this project.

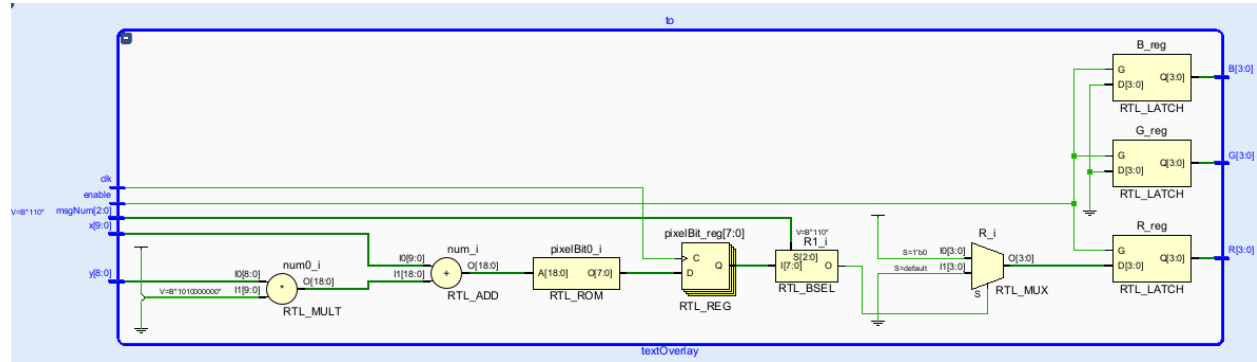


Figure 6: Overlay Logic

4.4 Collision Detection

This module should determine whether the frog hits another game object or enters a new region, and pass the collision signal to other modules for further calculations and game control. Specifically, when the frog hits a car, falls into the water, or enters the home area, it would pass a collision signal to reset the frog's position and update the scores and the number of lives remaining.

The module takes the position (top left corner) and width of every game object (cars, logs, etc.) as well as the frog's current position as inputs, and it outputs a collision type signal, an onLog signal that is used for moving the frog along with logs, and a prev_frog signal that indicates whether a lily pad position is occupied.

The module is divided into the following subsections:

4.4.1 Zone Detect

The death and reaching home signals are calculated based on the current region, so we will first determine the region (road, safe, water, grass) that the frog is currently in based on its y-coordinates. Since the game is separated clearly by regions, we can compare the frog's position with the start and end coordinates of each region, which are specified by parameters, and determine what logic to use for other modules. The module outputs a 3-bit one-hot code for enabling and disabling the regional modules. Specifically, if we decide that the frog is in some region i , we set the i th bit to 1 and others to 0, and each bit is then connected to the reset port on the module responsible for that area.

4.4.2 Road

This module detects whether the frog dies in the road region. It takes the frog position and the car positions and widths as inputs. For each car car_i , we check for its overlap with the frog, $hit[i]$. If there is one or more overlaps, it means that the frog has hit a car, and we output a one-bit active high death signal, that is, $death = \sum_i hit[i]$.

⁷See Nandland; Xilinx, Inc., 7 Series FPGAs Memory Resources User Guide (UG473)

4.4.3 Water

This module detects whether the frog dies in the water region. It takes the frog position and the log positions and widths as inputs. For each log log_i , we check for its overlap with the frog $hit[i]$, the same way we did with the road death. If there is no overlap, it means that the frog falls into the water, and we output a one-bit active high death signal, that is, $death = \overline{\sum_i hit[i]}$

4.4.4 Grass

This module detects whether the frog dies or reaches home in the grass region. The inputs are the frog's position and the previous frogs that reached home, and the output is a two-bit signal where one bit indicates death and the other indicates home.

The home positions are specified using a margin size and a home width parameter. For home number i , the rectangular home area can be defined with the top left corner coordinates $(\delta + i * W_{home})$ where δ is the margin size of the leftmost home to the screen and W_{home} is the width of the home area. Therefore, we can check the frog overlap with each home using the same logic for road and water region, and if overlap exists, a home signal is sent, and if it doesn't exist, the death signal is sent.

4.4.5 Previous Frogs

We also need to take into account the previous frogs that went home. The i th bit of the $prev_frog$ signal is high if there is a frog there, and low otherwise. So we check whether the prev frog signal has a one on the overlap home index, which is just $prev_frog \& hit$. When the frog actually reaches home, we want to update the prev frog signal to record that change. To add in the new frog, we can just OR the incoming hit home signal with the previous frogs and store it back in to the flip flop.

4.4.6 Check Overlap

This is the module that the above modules instantiates that checks for collisions. The module takes in the frog's position and the game object's position and width as inputs and outputs a one-bit active high overlap signal. The frog width, height, and the object height are fixed in this game to be exactly 32. Since the shape of the frog doesn't fill the box, the boundary between hit and miss that the user might consider reasonable should be slightly smaller than the actual sprite. So we draw the collision box with a collision margin δ of 8-10. The detailed descriptions are shown below in figure 7.

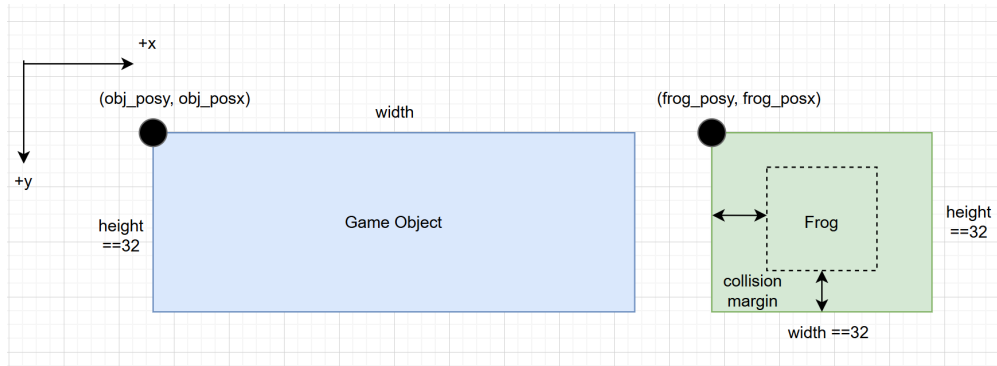


Figure 7: Check Overlap Logic

The coordinate of the top left of the collision box is $(x_{col1}, y_{col1}) = (x_{frog} + \delta, y_{frog} + \delta)$, and that of the top right corner is $(x_{col2}, y_{col2}) = (x_{frog} + W_{frog} - \delta, y_{frog} + \delta)$ where $W_{frog} = 32$ is the width of the frog. We determined that the frog collided with the object if and only if $y_{col1} = y_{col2}$ is in between y_{obj} and $y_{obj} + H_{obj}$ where $H_{obj} = 32$ is height of the object, and at the same time either x_{col1} or x_{col2} is in between x_{obj} and $x_{obj} + W_{obj}$.

4.4.7 Testing

We performed unit testing on all the submodules in this section, as well as the entire module. For the test benches, we decreased the grid size to 20 by 15 and focused on one "frame", meaning that all the game objects are fixed at one position. Then we validated our collision code by traversing the frog through the whole grid. After that, we tested on the screen by expanding the grid dimensions by toggling the screen width/height parameters and added in the movement of game objects.

4.5 Game Logic

4.5.1 Overview

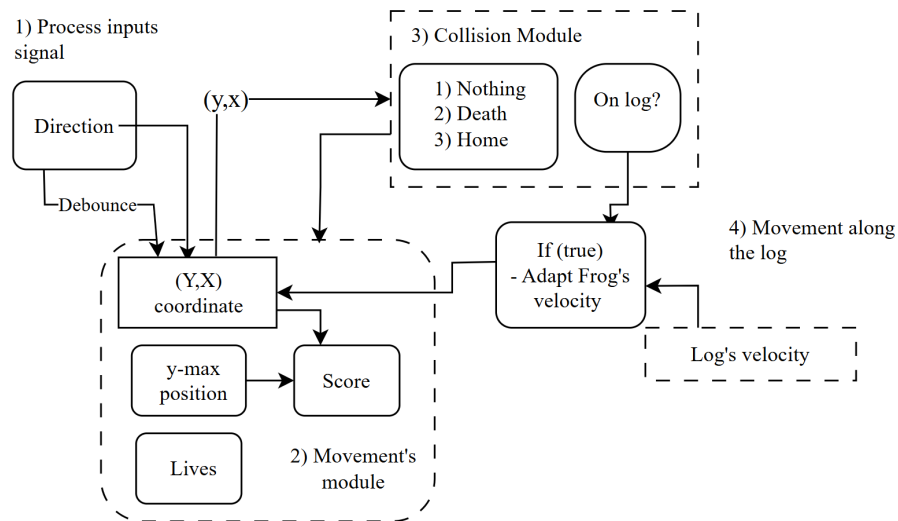


Figure 8: Overview of movement's module interaction

4.5.2 Debounced Input and Sampled signal

Buttons for inputs are prone to rapid toggle between on and off state, that may sample multiple signals. To prevent this problem, inputs are passed through a debouncer that only starts to sample input after the signal stayed high for 2 milliseconds. For each input to only be processed as a signal movement, the debounced signals are then sampled as a single signal on a negative edge of the input signal.

4.5.3 Interaction with collision module

Interactions of the frogs are classified into Nothing, Death and Home, which are sent as a 2-bit binary encoded signal from collision module.

1) Nothing: this is the default signal from collision module. The frog position is determined by combinational logic, and score increments by 10 as the frog moves up. The position is updated during the clock's sampling. A register is also dedicated to keep track of maximum y-position reached by the frog per round. This prevents the score from increment when the frog moves back to the previous reached position.

2) Death: Each frog has three lives. For every flag, the frog is reset to origin's and lives decrement by one. The death signal is passed to a timer module to reset the timer for the next frog.

3) Home: A reward of 100 points per frog arrived home, and a new frog entered.

4.5.4 Movement along the log

The upper half of the rows, the frogs also depends on the drifting movement with the log's speed. Each row moves by different amount of pixel. Therefore, the frog's drifting speed is processed through checking for its y position, from which the row of that speed can be used and taken into calculation of x-current every clock cycle. In figure 3, The number indicate the number of pixels that the frog needed to be increment by when the frog is on the log or turtle.

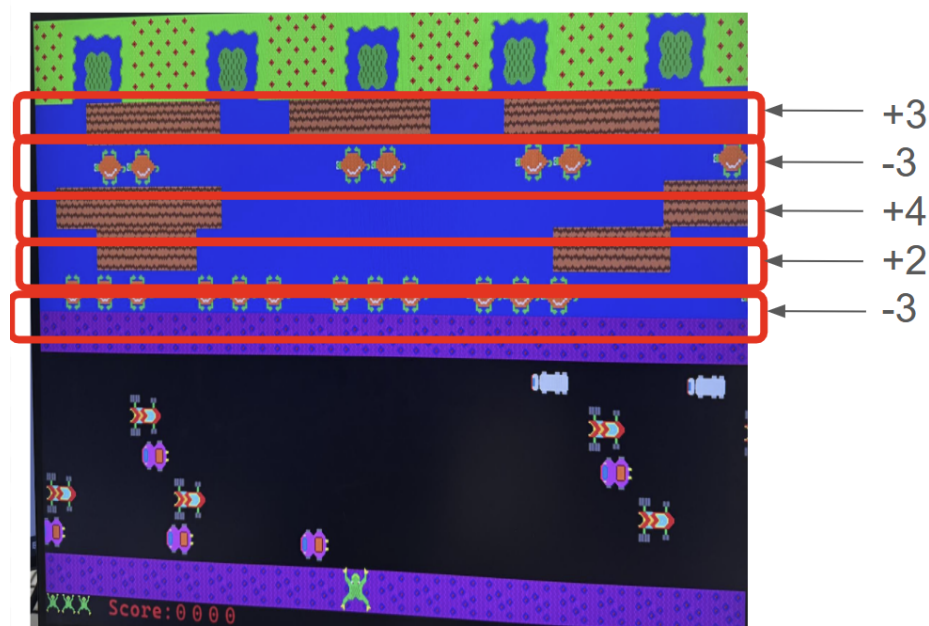


Figure 9: Classified y-position to account for frog's x position to move along the log's speed

4.5.5 Timer and Communication with VGA's module

Using clock divider, 1 Hz clock signal is used to control both the timer displayed as a countdown and timing for text overlayed during introduction, game over.

The module classified the game into Finite state machine (FSM). During the states's display, the enable to VGA's text overlay is pulled high with instructional display's binary encoded for the text needed to input. The timer module, is control the time the text needed to be display.

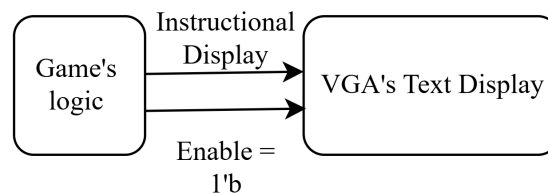


Figure 10: Output's to VGA's module

5 Conclusion

Based on the observations made throughout this project, it is evident that by employing the proper VGA color correction techniques and layer priority selection logic, using BRAM and Vivado .mem files effectively, and using well-designed collision logic and FSMs to control game mechanics, it was possible to create a high-quality FPGA derivation of Frogger. The VGA display was successfully implemented, allowing the screen to display all the objects appropriately. Despite initial timing issues to produce VGA colors, the solution was achieved, and a successfully wrapping effect was embedded into the final design. Based on these results, the system is able to take many complex logic, and implement it together in hardware design accurately and efficiently. Additionally, through using masking and efficient ROM access strategies, it was possible to significantly reduce the amount of memory necessary to store game assets on board the FPGA, and with the use of BRAM, entire full-screen text overlays could be included without placing excessive demand on LUTs and Flip-Flops to store all the required data. The use of state machines to drive overlays and control game logic also serves as a means of applying course knowledge to simplify what would otherwise be a very challenging problem into a finite set of known states. Furthermore, given the very low power draw of the FPGA itself, this project demonstrates the extreme efficiency of custom-built logic relative to software solutions, an important distinction in our ever-more environmentally conscious world. As such, the implications of this project extend far beyond the realm of pedagogy, providing a tangible example of the multifarious benefits of designing efficient and purpose-built hardware in an increasingly software-dependent world.

Works Cited

- Boston University. ENG EC 311: Introduction to Logic Design. 2025, Accessed: 2025-12-10.
- Digilent Inc. Nexys 4 DDR FPGA Board Reference Manual. rev. C, Accessed: 2025-12-07, Apr. 2016, digilent.com/reference/programmable-logic/nexys-4-ddr/reference-manual.
- Green, Will. “Video Timings: VGA, 720p, 1080p”, June 2020, Accessed: 2025-12-10, projectf.io/posts/video-timings-vga-720p-1080p/.
- Nandland. “Lesson 15: What is a Block RAM (BRAM)?”, 2022, nandland.com/lesson-15-what-is-a-block-ram-bram/. Accessed 10 Dec. 2025.
- Patel, Sagar, and Shalini Mishra. “Design and Implementation of VGA Controller on FPGA”. *International Journal of Research and Analytical Reviews (IJRAR)*, vol. 6, no. 2, Apr. 2019, Accessed: 2025-12-07, pp. 712–18. www.ijrar.com/upload_issue/ijrar_issue_20543524.pdf.
- Project F. “Hardware Sprites”, 2020, projectf.io/posts/hardware-sprites/. Accessed 3 Dec. 2025.
- Xilinx, Inc. 7 Series FPGAs Memory Resources User Guide (UG473). 2016, docs.amd.com/v/u/en-US/ug473_7Series_Memory_Resources.
- . Vivado Design Suite User Guide: Synthesis (UG901, v2022.2). Accessed: 2025-12-09, Nov. 2022, pp. 150–60 , 241, www.xilinx.com/support/documents/sw_manuals/xilinx2022_2/ug901-vivado-synthesis.pdf.